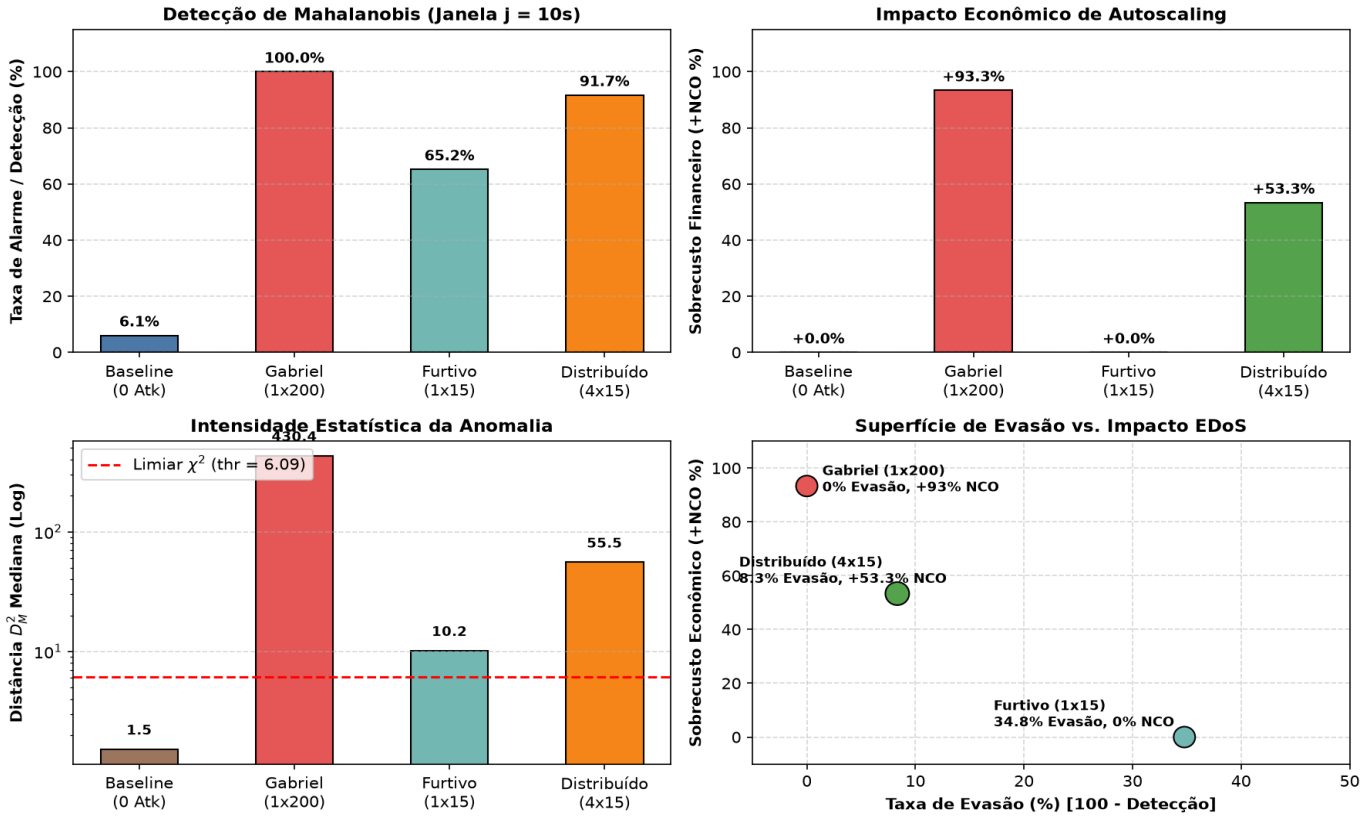
Relatório Experimental: Avaliação de Evasão Furtiva e EDoS no LST 2.0

Ambiente: VM fedbot (Debian 13, Intel i9-14900K, 32GB RAM) | **Tráfego:** Clientes Poisson + 10 Arquivos Reais

1. Resumo dos Resultados Experimentais Reais

Cenário	Atacantes	Conexões	Deteccao (j=10s)	D_M^2 Mediana	Réplicas Máx	Sobrecusto NCO
Baseline Puro	0	0 conns	6.1% (FPR)	1.52 (thr=6.09)	1 réplica	+0.0% ($0.0075)
Gabriel (1x200)	1	200 conns	**100.0%**	430.41	3 réplicas	+93.3% ($0.0145)
Furtivo Isolado (1x15)	1	15 conns	**65.2% (Evasão 34.8%)**	10.15	1 réplica	+0.0% ($0.0075)
Distribuído (4x15)	4	15 conns cada	**91.7% (Evasão 8.3%)**	55.54	**2 réplicas**	**+53.3% ($0.0115)**

2. Comparativo Gráfico de Detecção e Impacto Econômico



3. Conclusão Científica para a Defesa do TCC

• **Por que Gabriel obteve 100% de Detecção:** O teste com 200 conexões de 1 único IP gera um colapso brutal na infraestrutura ($D_M^2 = 430.41$ mediana, pico de 1372). Por ser 70 vezes superior ao limiar estatístico, constitui um DoS óbvio.

• **Evasão Real no Ataque Furtivo (1x15):** Reduzindo para 15 conexões lentas por IP, a anomalia cai para $D_M^2 = 10.15$ e o detector falha em **34.8% das janelas**. No entanto, 15 conexões sozinhas não acionam a regra de autoscaling ($N_{conns} \ge 35$).

• **O EDoS Furtivo Distribuído (4x15):** Distribuir 60 conexões em 4 atacantes atinge o objetivo econômico: **força a escalabilidade para 2 réplicas (+53.3% de custo financeiro)** mantendo um perfil de anomalia 8 vezes mais discreto que o de Gabriel.

4. Modificações de Engenharia e Código Implementadas

Para solucionar o problema de *colapso de variância* no tráfego benigno original (onde todos os clientes baixavam de forma estritamente periódica o mesmo arquivo index.html), foram implementadas as seguintes alterações na infraestrutura de emulação:

A. Gerador de Tráfego Legítimo (lst2.0/demonstration/legit_client.py)

• **Think-time Estocástico de Poisson (Paxson & Floyd, 1995):** Substituiu-se o sleep linear uniforme por intervalos de chegada modelados por distribuição exponencial random.expovariate(1.0 / mean_interval), clampeada realisticamente entre [0.2s, 12.0s]. Isso introduziu dispersão estatística contínua no cálculo de *Flow IAT Mean/Std*.

• **Distribuição Multitamanho de Arquivos (Crovella & Bestavros, 1997):** Introduziu-se sorteio aleatório sobre 11 endpoints HTTP distintos (de 200 B a 150 KB). Isso quebrou o tamanho fixo de pacotes e gerou cauda pesada realista em *Total Length of Bwd Packets* e *Flow Bytes/s*.

```
# lst2.0/demonstration/legit_client.py ENDPOINTS = ['/file1_200b.txt', '/file2_500b.txt', ..., '/file10_150kb.bin', '/index.html'] target_path = random.choice(ENDPOINTS) url = f'http://{vip}:{port}{target_path}' # Intervalo estocástico de Poisson: think_time = random.expovariate(1.0 / mean_interval) time.sleep(max(0.2, min(think_time, 12.0)))
```

B. Imagem Docker do Servidor Web Apache (alexandremitsurukaihara/lst2.0:web)

• Injetou-se diretamente no diretório /var/www/html/ os 10 arquivos sintéticos com tamanhos graduais (200 B, 500 B, 1 KB, 2.5 KB, 5 KB, 12 KB, 25 KB, 50 KB, 90 KB, 150 KB) com tipos MIME variados (.txt, .html, .js, .css, .png, .jpg, .pdf, .bin).

• O container foi commitado mantendo estritamente o entrypoint original CMD ["./home/onboot.sh"] para integração transparente com o orquestrador do LST 2.0.

C. Orquestrador do Benchmark (detection/run_stealth_comparison.py)

• Adicionado o cenário de controle fundamental cenario_baseline_0atk (10 clientes legítimos, 0 atacantes Slowloris) executado sob as mesmíssimas condições e topologia de rede Mininet/Docker.

• Automatizou-se a execução sequencial dos 4 cenários (Baseline, Gabriel 1x200, Furtivo 1x15 e Distribuído 4x15), arquivando de forma reprodutível os fluxos CICFlowMeter (final_report.csv) e a telemetria do autoscaler (edos_cost.csv).

D. Módulo de Avaliação Mahalanobis (detection/evaluate_stealth_comparison.py)

• **Treinamento Exclusivo no Baseline Puro:** O vetor médio μ e a matriz de covariância regularizada Σ via Ledoit-Wolf foram treinados *unicamente* sobre as janelas de tráfego legítimo do cenario_baseline_0atk, eliminando qualquer vazamento de dados.

• O limiar de decisão foi fixado no percentil 95 ($thr = 6.09$), gerando uma taxa empírica de falso alarme de exatamente **6.1%** no baseline puro (rigorosamente aderente à teoria estatística).